

Concurrency Interview Questions

Thread, Mutex, Race Condition, Deadlock, Atomic — ամբողջական ժողովածու

Այս ֆայլը concurrency-ի interview հարձերի zhoxovacu e code orinaknerov (sxal + chisht lucum). Erek mas.

Մաս A — 30 multiple-choice. Մաս B — 25 bac (open) harc. Մաս C — 15 code / "inchne xndiry, inchpes lucel".

Bolor patasxannery **miayn verjum** en (Answer Key A, B, C).

ՄԱՍ A — Multiple Choice (30)

Q1

Thread inch e? - (a) arancin program - (b) program-i amenapoqr kataroghakan miavor - (c) memory block - (d) file

Q2

Process vs thread - memory? - (a) process shared, thread arancin - (b) process arancin, thread shared - (c) erku-n el shared - (d) erku-n el arancin

Q3

Vory-n e aveli tep steghcel? - (a) process - (b) thread - (c) nuyny - (d) ochmeky

Q4

std::thread-ի join() inch e anum? - (a) thread jnjum - (b) spasum minchev thread avartvi - (c) nor thread - (d) detach

Q5

detach() inch e? - (a) spasum - (b) thread darnum ankax (background) - (c) jnjum - (d) lock

Q6

thread object ochnceluc araj chka join/detach -> ? - (a) OK - (b) std::terminate - (c) leak - (d) deadlock

Q7

Mutex inch e? - (a) memory - (b) Mutual Exclusion - mek thread miangamic critical section - (c) thread - (d) pointer

Q8

Mutex inch problem e lucum? - (a) deadlock - (b) race condition - (c) leak - (d) crash

Q9

lock_guard-i aravelutyuny? - (a) arag - (b) RAII - lock ctor/unlock dtor (exception-safe) - (c) qich memory - (d) atomic

Q10

lock_guard vs unique_lock? - (a) nuyny - (b) unique_lock chkun (dzerqov lock/unlock, condition_variable) - (c) lock_guard chkun - (d) unique_lock hin

Q11

Race condition inch e? - (a) thread arag - (b) mi qani thread nuyn data poxum, ardyunq kargic kaxvac - (c) deadlock - (d) leak

Q12

counter++ inchu e race? - (a) atomic e - (b) 3 qayl (read/+1/write) - thread-ner xaŋnvum - (c) dandaghn - (d) chka race

Q13

Race condition-i lucum? - (a) mutex / atomic - (b) aveli thread - (c) detach - (d) ochinch

Q14

Data race inch e? - (a) logikakan bug - (b) sync-ic aranc nuyn memory (write) -> UB - (c) deadlock - (d) leak

Q15

Race condition haytnaberog tool? - (a) valgrind memcheck - (b) ThreadSanitizer - (c) gprof - (d) gdb

Q16

Deadlock inch e? - (a) thread crash - (b) erku+ thread spasum irar anverjoren - (c) leak - (d) race

Q17

Deadlock-i patcha? - (a) nuyn kargov lock - (b) tarber kargov mi qani lock (circular wait) - (c) atomic - (d) detach

Q18

Coffman conditions-i qanaky? - (a) 2 - (b) 3 - (c) 4 - (d) 5

Q19

Deadlock-i amenapa?z lucum? - (a) aveli mutex - (b) misht nuyn kargov lock - (c) detach - (d) atomic

Q20

std::lock inch e anum? - (a) mek mutex - (b) mi qani mutex atomic (deadlock-free) - (c) unlock - (d) atomic

Q21

scoped_lock (C++17)? - (a) mek mutex RAI - (b) mi qani mutex RAI, deadlock-free - (c) atomic - (d) thread

Q22

atomic inch e? - (a) mutex - (b) anbjaneli operacia, race a?anc mutex - (c) thread - (d) lock

Q23

atomic vs mutex - erb atomic? - (a) bardz logic - (b) poqr operacia (counter/flag) - (c) container poxum - (d) misht mutex

Q24

atomic-i aravelutyuny mutex-i nkatmamb? - (a) bardz logic - (b) aveli arag (lock-free, chka blocking) - (c) qich memory - (d) sorted

Q25

compare_exchange (CAS) inch e? - (a) atomic add - (b) ete == expected -> set desired (lock-free himq) - (c) mutex - (d) load

Q26

recursive_mutex inch e? - (a) sovorakan - (b) nuyn thread mi qani lock - (c) atomic - (d) timed

Q27

Vory-n e blocking? - (a) atomic - (b) mutex lock (spasum) - (c) load - (d) store

Q28

atomic flag inchi hamar? - (a) counter - (b) stop/ready signal thread-neri mijev - (c) mutex - (d) lock

Q29

Critical section inch e? - (a) main - (b) kod vor miayn mek thread miangamic petq e mtni - (c) thread - (d) mutex

Q30

Thread-i himnakan aravelutyuny? - (a) qich memory - (b) parallelism (multi-core) + responsiveness - (c) sorted - (d) safety

ՄԱՍ B — Open Questions (25)

1. Thread inch e, process vs thread.
2. Thread inchpes es steghcum (std::thread), join/detach.
3. Inchu petq e join/detach thread ochnceluc araj.
4. Mutex inch e, inch problem e lucum.
5. lock_guard vs unique_lock.
6. Mutex-i tesaknery (recursive, timed, shared).

7. Race condition inch e, inchu e linum.
8. Inchu counter++ race e (nerhatuk).
9. Race condition inchpes es lucum.
10. Data race vs race condition.
11. Race condition inchpes es haytnaberum.
12. Deadlock inch e, inchu e linum.
13. Coffman 4 conditions.
14. Deadlock inchpes es xusapum (nuyn karg, std::lock).
15. std::lock / scoped_lock.
16. Atomic inch e, inchpes e lucum race-y.
17. Atomic vs mutex - erb vory.
18. compare_exchange (CAS).
19. Memory order (himunqner).
20. atomic flag-i ogtagorcum.
21. Critical section inch e.
22. Inchu multithreading dzhvar e (debugging).
23. Thread-safe inch e nshanakum.
24. Immutable data inchu e helpful concurrency-um.
25. Thread-local storage inch e.

УУУ C — Code: "inchne xndiry, inchpes lucel" (15)

C1

```
int counter = 0;
void work() { for(int i=0;i<100000;i++) counter++; }
std::thread t1(work), t2(work);
```

Inchne xndiry? - (a) ochmi - (b) race condition (counter shared, ne atomic) - (c) deadlock - (d) leak

C2

```
std::thread t(work);  
// join/detach chka
```

- (a) OK
- (b) std::terminate thread ochnceluc
- (c) leak
- (d) race

C3

```
std::mutex m1, m2;  
void A() { lock_guard l1(m1); lock_guard l2(m2); }  
void B() { lock_guard l1(m2); lock_guard l2(m1); }
```

- (a) OK
- (b) deadlock (tarber karg)
- (c) race
- (d) leak

C4 (C3-i lucum)

Inchpes lucel C3-y? - (a) aveli mutex - (b) nuyn kargov lock (erku-n el m1->m2) kam std::scoped_lock(m1,m2) - (c) detach - (d) atomic

C5

```
std::mutex m;  
void f() {  
    m.lock();  
    if (error) return; // <-- ?  
    m.unlock();  
}
```

- (a) OK
- (b) error-i depqum unlock chi hasni -> mutex zbaxvac (kam deadlock)
- (c) race
- (d) leak

C6 (C5-i lucum)

Inchpes lucel C5-y? - (a) dzerqov unlock amen path-um - (b) std::lock_guard (RAII, avtomat unlock) - (c) atomic - (d) detach

C7

```
std::atomic<int> counter{0};  
void work() { for(int i=0; i<100000; i++) counter++; }
```

- (a) race condition
- (b) OK (atomic, chka race)
- (c) deadlock
- (d) leak

C8

```
bool stop = false;           // sovorakan bool  
void worker() { while(!stop) doWork(); }  
void ctrl() { stop = true; }
```

- (a) OK
- (b) data race (stop shared, ne atomic/synchronized)
- (c) deadlock
- (d) leak

C9 (C8-i lucum)

- (a) mutex amen kardaluc
- (b) std::atomic stop
- (c) detach
- (d) ochinch

C10

```
std::vector<int> v;           // shared  
void add() { v.push_back(1); } // 2 thread
```

- (a) OK
- (b) race condition (vector ne thread-safe) -> mutex petq
- (c) deadlock

- (d) atomic bavakan

C11

```
std::recursive_mutex m;
void f() { lock_guard<recursive_mutex> l(m); f(); }
```

recursive_mutex-y inchu? - (a) sovorakan mutex -> deadlock recursion-i zham - (b) arag - (c) atomic - (d) leak

C12

```
std::thread t([](){ cout << "hi"; });
t.detach();
// main verjanum
```

- (a) misht "hi" tpvum
- (b) karo e chtpvel (main verjanum, detached thread chi hasni)
- (c) CE
- (d) deadlock

C13

```
std::atomic<int> a{5};
int exp = 5;
a.compare_exchange_strong(exp, 10);
cout << a;
```

- (a) 5
- (b) 10 (a==exp -> set 10)
- (c) 0
- (d) CE

C14

```
std::mutex m;
void f() { lock_guard l(m); longIO(); } // erkar I/O lock-i mej
```

Inchne xndiry? - (a) ochmi - (b) erkar lock -> ayl thread-ner spasum (performance) -- lock-y poqracu - (c) deadlock - (d) race

C15

```
std::scoped_lock lock(m1, m2); // C++17
```

- (a) deadlock hnaravor
- (b) deadlock-free mi qani mutex lock (RAII)
- (c) CE
- (d) miayn mek mutex

ANSWER KEY A

1-b · 2-b · 3-b · 4-b · 5-b · 6-b · 7-b · 8-b · 9-b · 10-b · 11-b · 12-b · 13-a · 14-b · 15-b · 16-b · 17-b · 18-c · 19-b · 20-b · 21-b · 22-b · 23-b · 24-b · 25-b · 26-b · 27-b · 28-b · 29-b · 30-b

ANSWER KEY B (hakirch)

1. Thread` program-i amenapoqr kataroghakan miavor. Process arancin memory (canr, isolated); thread shared memory (tep, mek process).
2. std::thread t(func); t.join() spasum; t.detach() ankax. Argument-ner` t(func, arg).
3. thread object ochnceluc araj petq e join/detach; hakaarak -> std::terminate.
4. Mutual Exclusion mek thread miangamic critical section. Lucum race condition.
5. lock_guard parz (chi bacvum, lock ctor/unlock dtor); unique_lock chkun (dzerqov lock/unlock, condition_variable).
6. mutex (sovorakan), recursive_mutex (nuyn thread mi qani lock), timed_mutex (try_lock_for), shared_mutex (read/write, C++17).
7. Mi qani thread nuyn shared data poxum; ardyunq kaxvac thread-neri kargic -> ankanxatesli.
8. counter++ 3 qayl (read/+1/write); thread-ner xarnvum -> lost update.
9. mutex (lock_guard) kam atomic; immutable/thread-local data.
10. Data race sync-ic aranc nuyn memory write -> UB. Race condition logikakan, kargic kaxvac.
11. ThreadSanitizer (-fsanitize=thread), Helgrind, code review.
12. Erku+ thread spasum irar anverjoren (kaxvac). Inchu` tarber kargov mi qani lock -> circular wait.

13. Mutual exclusion, hold and wait, no preemption, circular wait (bolor 4 petq en).
 14. Nuyn kargov lock; std::lock/scoped_lock; try_lock timeout.
 15. std::lock mi qani mutex atomic (deadlock-free); scoped_lock (C++17) RAI + std::lock.
 16. Anbajaneli operacia -> race aranc mutex, poqr operacia-neri hamar.
 17. atomic arag (lock-free), poqr operacia (counter/flag); mutex bardz critical section (blocking).
 18. compare-and-swap` ete arjeqy == expected, set desired atomic; lock-free himq.
 19. Atomic operacia-neri karganavorman erashxiq; default seq_cst (xist).
 20. Stop/ready signal thread-neri mijev, aranc mutex.
 21. Kod vor miayn mek thread miangamic petq e mtni (shared data poxum).
 22. Non-deterministic (kargy ankanxatesli), reproduce dzhvar, race/deadlock nurb.
 23. Thread-safe` mi qani thread-ic anvtang kanchvox (chka race/corruption).
 24. Immutable -> chka poxum -> chka race (kardal misht anvtang).
 25. Thread-local` amen thread ir arancin copy-n (thread_local keyword) -> chka sharing -> chka race.
-

ANSWER KEY C

C1-b · C2-b · C3-b · C4-b · C5-b · C6-b · C7-b · C8-b · C9-b · C10-b C11-a · C12-b · C13-b · C14-b · C15-b

Bacatrutyunner (C)

C1 counter shared + ne atomic -> race condition.

Lucum mutex/atomic. **C2** join/detach chka -> std::terminate. **C3** tarber kargov lock (A: m1->m2, B: m2->m1) -> deadlock. **C4** nuyn kargov lock kam scoped_lock(m1,m2) -> deadlock-free. **C5** error return -> unlock chi hasni -> mutex mnum zbaxvac. **C6** lock_guard (RAII) -> avtomat unlock amen path-um. **C7** atomic counter -> chka race -> OK. **C8** stop sovorakan bool shared -> data race. Lucum atomic. **C9** std::atomic stop. **C10** vector ne thread-safe -> push_back race -> mutex petq. **C11** recursion-i zham nuyn thread mi qani lock -> sovorakan mutex deadlock -> recursive_mutex. **C12** main verjanum -> detached thread karox e chhasni -> "hi" karox e chtpvel. **C13** a==exp(5) -> set 10 -> a=10. **C14** erkar I/O lock-i mej -> ayl thread-ner erkar spasum -> lock-y poqracru (I/O durs). **C15** scoped_lock mi qani mutex deadlock-free RAI.

Endhanur kanonner

THREAD: join/detach ochnceluc aʀaj | shared memory -> race vtang

MUTEX: lock_guard (RAII) | nuyn kargov lock (deadlock xusapel) | poqr critical section

RACE: shared + ne atomic -> mutex/atomic | ThreadSanitizer

DEADLOCK: tarber karg -> circular wait | nuyn karg / scoped_lock

ATOMIC: poqr operacia (counter/flag) arag | bardz -> mutex

vector/container NE thread-safe -> mutex petq